

STAT3888 自行车盗窃

时空基函数回归

从时空可视化到可信预测的完整项目手册

数据	bicycle.csv · 31,833条事件 · 140个社区 · 2014–2023
主线	Task 1 时空可视化 + Task 4 基函数线性回归
基础	从高等代数、概率论进入设计矩阵、样条、验证和诊断
版本	2026-07-27 · 基于真实数据验证结果

这本手册强调理解、复现与诚实验证。所有模型指标均来自真实的2023时间外测试集。

目录

章节按“核心路线—进阶加分—未来连接”组织。先完成核心路线，再根据时间深入。

目录	2
如何使用这本手册	4
项目要求、交付物与评分策略	4
四个Task的递进关系	4
Task 1：时空可视化是什么	5
Task 2：EOF分析是什么	5
Task 3：IDW预测是什么	6
Task 4：基函数线性回归是什么	6
为什么选择 bicycle.csv	7
研究问题、分析单位与目标变量	7
从高等代数和概率论进入模型	7
线性回归与设计矩阵	8
基函数：把一个变量变成一组可学习特征	8
B-spline：学习平滑的长期趋势	9
Fourier基函数：表示年度季节性	9
径向基函数：表示连续空间热点	9
社区指示基函数：学习稳定地点差异	10
Ridge正则化与偏差—方差权衡	10
为什么不能随机拆分时空数据	10
第0步：建立项目并准备 R 环境	11
第1步：读取 bicycle.csv 并理解 R 对象	11
第2步：审计日期、缺失、重复和季度	12
第3步：把事件表聚合为社区 × 月份	13
第4步：补齐零事件月份并验证面板	13
第5步：完成 Task 1 的四类时空图	14
第6步：按时间划分训练、验证和测试集	16
第7步：构造时空基函数与设计矩阵	16
第8步：建立两个简单预测基线	17
第9步：拟合 OLS 基函数模型	18
第10步：拟合 Ridge 并选择 λ	18
第11步：计算 MAE、RMSE 和 R^2	19
第12步：绘制预测与残差诊断	20
第13步：把分析压缩为四页 PowerPoint	21
第14步：从头复现并完成提交检查	21

真实模型结果与解释	22
预测图、残差图与模型诊断	23
四页PPT逐页设计	24
答辩问题与推荐回答	25
局限性、伦理与不可推广范围	25
从本项目走向未来数据科学	25
从开始到提交的最终检查清单	25
资料与继续学习入口	26

核心路线

如何使用这本手册

这不是一份“复制代码就交作业”的答案，而是一条从已有数学基础进入时空数据科学的完整路线。第一次阅读先理解研究问题和四个Task；第二次按操作章节运行R；第三次根据真实输出制作四页PPT。

- 核心路线：项目必须理解和执行。
- 进阶加分：用于体现方法深度，但必须建立在核心路线正确之上。
- 未来连接：帮助你把本项目迁移到GAM、树模型和时空机器学习，现在不要求全部掌握。

五个检查问题

面对任何公式或代码，先问：数据是什么形状？输入是什么？输出是什么？模型相信什么结构？怎样用未见数据证明结果可信？

核心路线

项目要求、交付物与评分策略

原项目要求从四个数据集中选择一个，完成必做的Task 1时空可视化，再从Task 2 EOF、Task 3 IDW、Task 4基函数线性回归中选择一个。最终提交最多四页PowerPoint。

时空可视化	30%	图回答明确问题；时间、空间、颜色尺度和单位完整
选修方法	40%	原理正确；验证无泄漏；与基线比较；诚实解释局限
表达与逻辑	30%	研究问题贯穿数据、方法、结果和结论

最重要的策略

不要因为方法名字高级就堆模型。评分最高的路线是：研究问题清楚、方法和数据匹配、验证方式可信、四页之内讲得明白。

核心路线

四个Task的递进关系

Task 1是所有路线的共同起点：先看见规律和问题。三个选修Task分别把探索发展为模式分解、空间插值或解释与预测。它们没有绝对的新旧高低，关键是是否回答了正确的问题。

Task 1	数据在什么地方、什么时候发生什么变化？	地图、时间序列、热力图
Task 2	哪些共同空间模式解释了大部分变化？	EOF载荷、PC时间序列、方差比例
Task 3	没有观测的位置可能是多少？	IDW插值面、交叉验证误差
Task 4	时空结构能否解释并预测结果？	模型、预测、残差和测试指标

本项目的选择

选择Task 1 + Task 4，因为bicycle数据具有十年时间、固定社区位置和明确的未来预测问题。

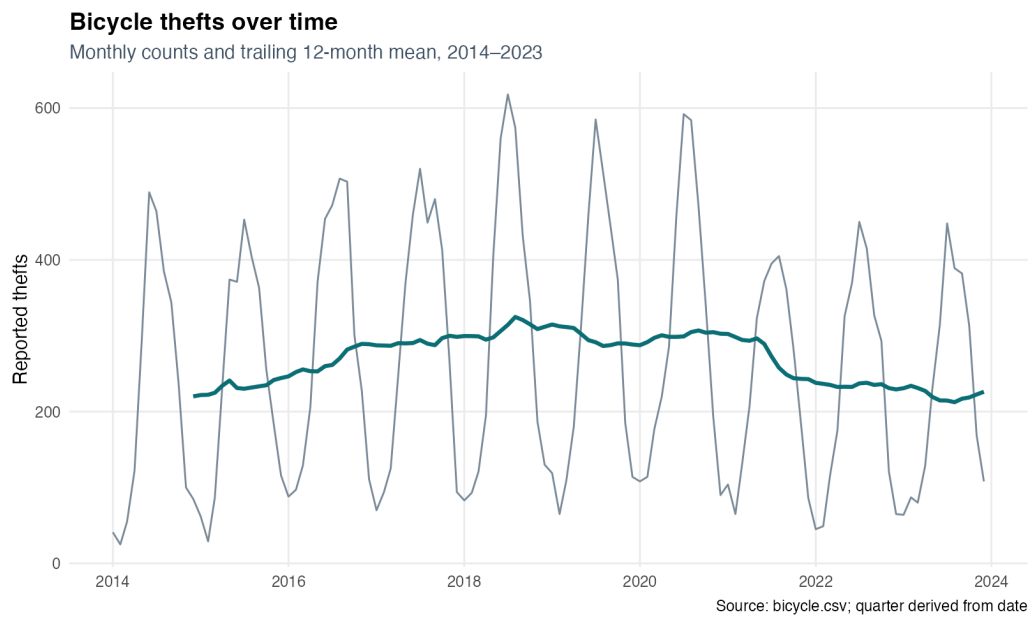
核心路线

Task 1：时空可视化是什么

时空可视化不是把经纬度画成点，而是同时说明“在哪里、什么时候、数值多大、怎样变化”。它用于发现热点、趋势、季节性、异常、缺失和尺度问题，也是后续模型设计的证据。

时间序列	总体数量怎样随时间变化？	每月盗窃数与12个月移动趋势
季节箱线图	同一月份跨年份是否稳定？	一月至十二月分布
空间热点图	长期高值集中在哪里？	社区中心点大小和颜色
时空热力图	地点模式怎样随时间同步变化？	前25社区 × 月份

- 比较多张地图时必须固定颜色范围。
- 0表示记录数为零；NA表示没有观测，两者不能混淆。
- 热点是描述性结果，不等于该社区本身导致盗窃。
- 经纬度是社区中心点，不是每起案件的准确位置。



Task 1 示例：总体月度趋势与12个月移动平均。

完整解释

Task 2：EOF分析是什么

EOF把“时间 × 地点” 矩阵压缩为少数主要空间模式和对应的的时间变化。它等价于对中心化的时空矩阵做PCA/SVD，最适合规则、较完整的空间面板。

X的行是时间、列是地点；V的每一列给出一个EOF空间模式；U Σ 给出对应PC时间序列； Σ 中的奇异值决定解释方差比例。

- 中心化：减去每个地点的时间均值，关注偏离正常水平的共同变化。
- 标准化：只有在希望每个地点拥有相同权重时使用；否则会放大低方差地点。

- EOF正负号可以整体翻转，空间图和PC同时翻转后含义不变。
- EOF1解释方差最大，不等于因果最重要，也不自动具备预测能力。

适合的数据

COVID地点×日期或汇总后的bicycle社区×月份都可构造矩阵；缺失严重的Big Mac面板需要先处理缺期。

完整解释

Task 3 : IDW预测是什么

IDW (Inverse Distance Weighting) 根据附近已观测地点预测未观测地点。核心假设是空间上越近的地点越相似，因此距离较近的观测获得更大权重。

s 是目标地点， s_i 是已观测地点， d_i 是距离， p 控制距离衰减速度。 p 越大，预测越依赖最近地点。

- 必须留出部分已知地点，假装它们未知，再用RMSE/MAE评价。
- p 和邻居数量应通过交叉验证选择，而不是凭感觉。
- 大范围经纬度不能简单当平面坐标距离；需要投影或球面距离。
- IDW通常不给可靠不确定性，也不使用协方差结构。
- 自行车盗窃可能受商业区、交通和暴露量影响，不一定只由距离决定，因此不作为本项目主方法。

核心路线

Task 4 : 基函数线性回归是什么

普通直线只能表示固定斜率。基函数先把时间和空间坐标转换成一组更丰富的特征，再对这些特征做线性回归。模型对输入可以是弯曲、周期和局部的，但对待估系数仍然是线性的。

y 是社区 s 在月份 t 的盗窃数。 B_{trend} 表示长期趋势， B_{season} 表示季节周期， B_{space} 表示空间和社区差异。

B-spline	平滑且可局部弯曲	十年长期趋势
Fourier	周期性的正弦和余弦	每12个月重复的季节性
径向基函数	围绕空间中心逐渐衰减	连续空间热点
社区指示基函数	每个社区独立基线	稳定的社区差异

为什么选它

它严格符合Task 4，同时训练特征工程、设计矩阵、验证、正则化、诊断和解释，能够连接未来的数据科学学习。

核心路线

为什么选择 bicycle.csv

bicycle	31,833事件；140社区；10年；无缺失	重复行无ID；坐标为社区中心	最适合
covid	规则日期；可直接形成面板	仅51天；累计值修订；地理层级混杂	次选
bigmac	全球题材；连续价格	面板完整率约70.7%；日期歧义	清洗成本高
ufo	题材醒目；有城市坐标	日期歧义；报告偏差；事件稀疏	不推荐

bicycle的固定社区位置允许学习空间结构，120个月允许学习季节和长期趋势。汇总为社区×月份后，共有16,800个分析单元，足以比较简单基线与基函数模型。

不分析bike_cost的原因

核心问题是盗窃数量的时空变化。bike_cost包含1,623个非正值和少量极端值，加入会改变研究问题并增加不必要清洗。

核心路线

研究问题、分析单位与目标变量

研究问题：时空基函数能否预测不同社区的自行车盗窃数量，并揭示盗窃活动随空间、季节和长期时间的变化？

- 原始粒度：一行是一条被报告的盗窃事件。
- 建模粒度：一行是一个社区在一个月份的盗窃数量。
- 空间单位：140个社区及其中心经纬度。
- 时间单位：2014年1月至2023年12月，共120个月。
- 响应变量：theft_count；为减弱右偏，模型使用 $\log(1+\text{count})$ 。

解释边界

没有人口、自行车拥有量、通勤量或报案概率，因此结果描述的是“报告数量”，不能称为个人风险或因果效应。

核心路线

从高等代数和概率论进入模型

你已经学过的向量、矩阵、线性变换、期望、方差和协方差不是旁支，而是这个项目的语言。每个社区月份形成一行特征向量，所有行叠成设计矩阵 X ；系数向量 β 表示每个特征的权重。

X 是 $n \times p$ 设计矩阵， $n=16,800$ 个社区月份， p 是基函数个数； β 是 $p \times 1$ 系数； ϵ 是模型未解释的随机误差。

- 矩阵乘法：把多个基函数特征按系数组合成预测。
- 秩与共线性：列过于相似时，系数不稳定。
- 期望：模型预测可理解为给定特征后的平均水平。
- 方差：衡量结果波动，也连接到RMSE和偏差-方差权衡。
- 协方差：解释相邻月份或相邻地点为什么可能具有相关误差。

核心路线

线性回归与设计矩阵

“线性”指模型对系数 β 是线性的，而不要求输入变量只能原样进入。只要先计算好特征列，平方、正弦、样条或空间核都能成为 X 的一列。

最小二乘选择使残差平方和最小的系数。残差是实际值减预测值，不是原数据的误差标签。

y	16.800×1	每个社区月份的 $\log(1+\text{盗窃数})$
X	$16.800 \times p$	时间、季节、空间和社区基函数
β	$p \times 1$	每一列基函数的待估权重
$\hat{y}$	16.800×1	模型预测，再用 $\exp(\hat{y})-1$ 还原

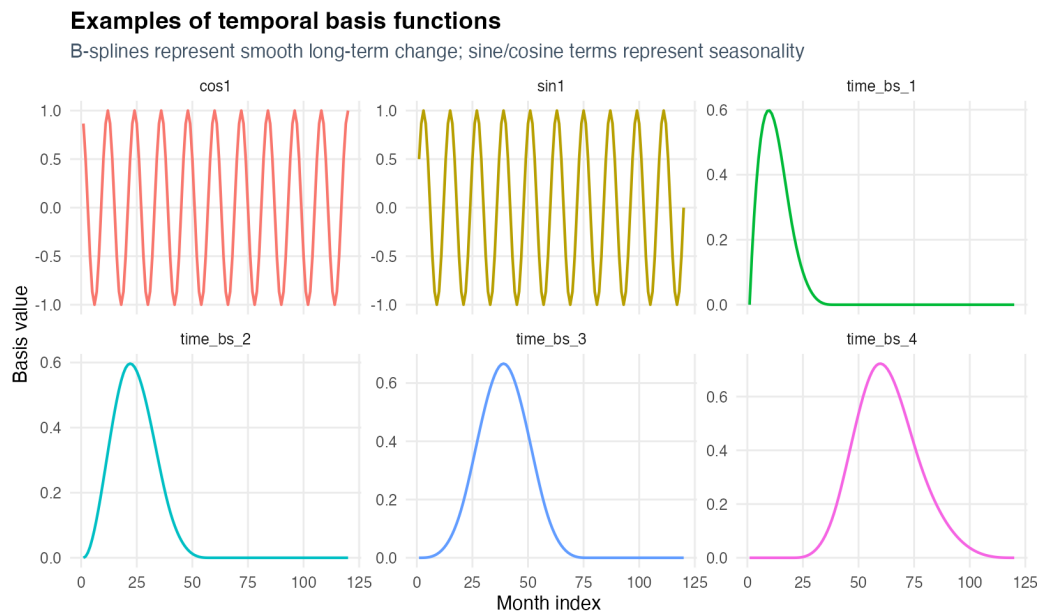
核心路线

基函数：把一个变量变成一组可学习特征

基函数 $\phi(x), \dots, \phi_K(x)$ 像一组积木。每一块有固定形状，模型学习怎样加权组合。这把“拟合一条复杂曲线”转化成“估计一组线性系数”。

ϕ_k 由我们选择， β_k 由数据估计。K太少会欠拟合，K太多可能过拟合，因此需要验证或正则化。

- 好处：非线性仍能使用成熟的线性代数和回归工具。
- 代价：基函数类型、数量和节点位置成为新的建模选择。
- 原则：每类基函数对应明确结构，而不是无限添加列。



项目中的B-spline与Fourier基函数示例。

核心路线

B-spline：学习平滑的长期趋势

高次多项式在整个区间相互牵动，边界容易剧烈震荡。B-spline把时间区间分成若干段，每个基函数只在局部区域显著，因此既能弯曲又相对稳定。节点knots决定不同多项式片段连接的位置。

R代码

```
time_basis <- splines::bs(
  panel$time_index,
  knots = c(20, 39, 58, 77),
  Boundary.knots = c(1, 120),
  degree = 3
)
```

- degree=3表示三次分段多项式。
- 节点从训练期确定；边界使用已知的项目月份范围，不使用未来盗窃值。
- 模型学习基函数权重，不直接把单个系数解释为某月增长率。
- 长期趋势图应展示组合后的效应，而不是逐个系数。

核心路线

Fourier基函数：表示年度季节性

自行车盗窃具有明显年度周期。月份类别需要11个参数且在十二月与一月之间不连续；正弦和余弦天然首尾相接，用少量参数表示平滑周期。

m是一月至十二月。第一对表示一年一次的波动，第二对允许一年内出现更复杂的双峰或不对称形状。

R代码

```
sin1 <- sin(2*pi*month_of_year/12)
cos1 <- cos(2*pi*month_of_year/12)
sin2 <- sin(4*pi*month_of_year/12)
cos2 <- cos(4*pi*month_of_year/12)
```

与概率论的连接

这里不是假设盗窃服从正弦分布，而是把周期函数当作确定的特征列；随机性仍留在误差 ϵ 中。

核心路线

径向基函数：表示连续空间热点

径向基函数以某个空间中心为峰值，距离越远取值越小。多个中心的函数叠加后，可以形成平滑的空间表面。它比直接加入经纬度直线项更能表达局部热点。

s是社区坐标， ck 是第 k 个空间中心， σ 控制影响范围。 σ 太小会形成尖峰，太大会把城市抹平。

- 本项目从社区中心中均匀选取16个RBF中心。
- 经纬度先中心化并按共同尺度缩放，避免数值单位影响。
- RBF表达连续位置，但坐标仍只是社区中心。
- 如果目标是新城市或城市外推，当前基函数不保证可靠。

进阶加分

社区指示基函数：学习稳定地点差异

第一版只使用连续空间RBF时，模型没有超过社区历史均值。原因是两个经纬度接近的社区仍可能因土地用途、商业活动或报案习惯而长期不同。于是加入每个社区的一系列0/1指示基函数。

如果观测属于社区 j ，该列为1，否则为0。这允许模型学习每个社区的稳定基线，同时保留季节和长期趋势。

重要取舍

社区指示基函数提升已知社区的预测，但无法直接预测训练中从未出现的新社区。因此它服务于“预测2023年现有140社区”，空间留出时应另用RBF-only模型做敏感性检查。

进阶加分

Ridge正则化与偏差一方差权衡

基函数增多后，部分列可能高度相关。Ridge在残差平方和之外惩罚大系数，使解更稳定。惩罚强度 λ 必须用验证集选择，不能看测试集后再调整。

$\lambda=0$ 退化为普通最小二乘； λ 越大，系数越接近0，模型越平滑但偏差可能增加。

真实结果中，2022验证选择的 λ 约为0.0001，接近无惩罚；2023测试RMSE中Basis OLS为2.05、Basis Ridge为2.06。这说明正则化作为稳健性工具有价值，但不能宣称它在本数据上优于OLS。

现代数据科学不是强迫复杂模型胜出

正确结论是：验证决定复杂度，测试集只做一次最终评价；如果简单模型更好，就选择或报告简单模型。

核心路线

为什么不能随机拆分时空数据

随机拆行会让同一社区的2023年1月进入训练、2月进入测试，模型等于提前看到了未来附近的信息。这会低估真实部署误差，属于数据泄漏。

训练集	2014–2021	估计模型与基础结构
验证集	2022	选择 λ 等超参数
测试集	2023	一次性报告最终泛化性能

- 时间验证始终用过去预测未来。
- 所有均值、节点、缩放参数和系数只能由训练数据得到。
- 测试集结果不好时不能返回修改模型再重测，否则测试集变成了验证集。
- 未来可使用rolling/sliding validation观察不同时段稳定性。

核心路径· 照做与理解

第0步：建立项目并准备 R 环境

这一步解决什么问题	让 R 知道项目文件在哪里，并让后续脚本能找到所需函数。工作目录错误是初学者最常见、也最容易误以为是代码错误的问题。
为什么现在运行	所有相对路径都以当前工作目录为起点。先固定项目根目录，后续才能稳定读取数据、保存图片 and 重复运行分析。
输入对象	一个新文件夹，例如 `stat3888_bicycle_project/`；其中建立 `data/`、`R/`、`output/figures/` 和 `output/analysis/`。

在 RStudio Console 中运行

```
getwd()
dir.create("data", showWarnings = FALSE)
dir.create("output/figures", recursive = TRUE, showWarnings = FALSE)
install.packages(c("tidyverse", "lubridate", "glmnet"))
library(tidyverse)
library(lubridate)
library(glmnet)
```

逐行解释	`getwd()` 返回当前工作目录；`dir.create()` 建立文件夹，`recursive = TRUE` 会连同缺失的上层目录一起建立，`showWarnings = FALSE` 只隐藏“文件夹已存在”的提示。`install.packages()` 把软件包安装到电脑，通常只需执行一次；`library()` 把包加载到当前 R 会话，每次重新打开 RStudio 都要运行。安装和加载是两件不同的事。
------	--

运行后应该看到	`getwd()` 显示项目根目录；Environment 中没有报错；三个软件包可以正常加载。安装时出现下载信息属于正常输出。
---------	--

检查命令

```
getwd()
all(c("tidyverse", "lubridate", "glmnet") %in% rownames(installed.packages()))
```

常见错误	若出现 `there is no package called ...`，说明只加载但没有成功安装；重新运行 `install.packages()`。若之后提示找不到 `data/bicycle.csv`，先检查 `getwd()`，不要反复改分析代码。推荐在 RStudio 中用 File → New Project 建立 `.Rproj`。
------	--

评分连接	可复现性与软件说明会影响方法可信度。展示时应能说清 R、RStudio 和各软件包分别负责什么。
------	--

核心路径· 照做与理解

第1步：读取 bicycle.csv 并理解 R 对象

这一步解决什么问题	把磁盘上的 CSV 转换成 R 内存中的数据框 `bike`，并确认行数、列数和变量类型与预期一致。
为什么现在运行	后续所有清洗、作图和建模都依赖 `bike`。如果读错文件或字段类型错误，后面的漂亮图表也没有意义。
输入对象	`data/bicycle.csv`。它是事件表：每行代表一条被报告的自行车盗窃事件，而不是一个社区或一个月份。

在 RStudio Console 中运行

```
bike <- readr::read_csv(
  "data/bicycle.csv",
  show_col_types = FALSE
)
dim(bike)
dplyr::glimpse(bike)
```

逐行解释

``bike <-`` 把右侧结果保存为名为 ``bike`` 的对象；``<-`` 可以理解为“把结果交给这个名字”。``readr::read_csv()`` 中的 ``readr::`` 明确指定函数来自 `readr` 包，即使没有先运行 ``library(readr)`` 也能调用。引号内是相对于 ``getwd()`` 的路径。``show_col_types = FALSE`` 只关闭列类型提示，不改变数据内容。``dim()`` 返回“行数、列数”；``glimpse()`` 用紧凑方式展示列名、类型和样例。

运行后应该看到

``dim(bike)`` 应得到 ``31833 8``；``glimpse()`` 应显示 `date`、`quarter`、`day_of_week`、`neighborhood`、`bike_cost`、`location`、`long`、`lat`。`

检查命令

```
stopifnot(nrow(bike) == 31833)
stopifnot(ncol(bike) == 8)
names(bike)
```

常见错误

``cannot open file`` 或 ``file does not exist`` 几乎总是路径问题：确认文件确实放入 ``data/``，再检查 ``getwd()``。若行数不是 31.833，不要继续；先确认没有打开另一个同名文件，也没有在 Excel 中另存时损坏分隔符。

评分连接

明确数据来源、观测单位和真实样本量，是数据介绍与方法合理性的基础证据。

核心路径·照做与理解

第2步：审计日期、缺失、重复和季度

这一步解决什么问题

在改变数据前建立质量证据：日期能否解析、是否缺失、是否有完全相同行，以及原 `quarter` 字段是否与 `date` 一致。

为什么现在运行

审计必须先于删除或修正。否则无法区分原始问题和自己加工时制造的问题，也无法在报告中解释清洗决定。

输入对象

``bike``：31.833 × 8 的原始事件表。这里不使用未来模型结果，也不修改原始文件。

在 RStudio Console 中运行

```
bike <- bike |>
  dplyr::mutate(
    date = as.Date(date),
    derived_quarter = paste0(
      lubridate::year(date), "Q", lubridate::quarter(date)
    )
  )

sum(is.na(bike))
sum(duplicated(bike[, 1:8]))
dplyr::n_distinct(bike$neighborhood)
```

逐行解释

``|>`` 是 R 的管道：把左边对象作为右边函数的第一个输入，可按阅读顺序写处理流程。``mutate()`` 新增或修改列；``as.Date()`` 把文字转为日期；``year()`` 和 ``quarter()`` 从日期提取年份和季度；``paste0()`` 把它们拼成如 ``2014Q1``。``is.na()`` 标记缺失，``sum()`` 统计 TRUE；``duplicated()`` 标记与之前完全相同的行；``n_distinct()`` 统计不同社区。

运行后应该看到

原始字段无缺失；有 140 个社区；1,475 行与前面的原始八列完全相同；从 `date` 推导后可发现大量 `quarter` 不一致。

检查命令

```
audit <- tibble::tibble(
  missing_cells = sum(is.na(bike[, 1:8])),
  duplicate_rows = sum(duplicated(bike[, 1:8])),
  neighborhoods = dplyr::n_distinct(bike$neighborhood),
  first_date = min(bike$date),
  last_date = max(bike$date)
)
audit
```

常见错误

不要看到 1,475 就直接运行 `distinct()`。数据没有唯一事件 ID，同一日期和社区可能发生多起真实盗窃；自动删除会低估数量。若 `as.Date()` 产生 NA，先查看原始日期格式，再指定 `format=`，不能用猜测填补。

评分连接

高分清洗不是“删得多”，而是每个决定都有证据。报告应明确：重复行保留、quarter 从 date 重新推导。

核心路径·照做与理解

第3步：把事件表聚合为社区 × 月份

这一步解决什么问题

把“一行一事件”转换为“一行一社区月份”，响应变量 `theft\_count` 表示该社区该月报告了多少起盗窃。

为什么现在运行

Task 1 和 Task 4 都需要固定的时空分析单位。事件表中没有发生事件的月份根本没有行，不能直接与其他月份比较。

输入对象

`bike`，其中 date 已是 Date 类型，neighborhood 是社区名称。输出暂时只包含发生过事件的社区月份。

在 RStudio Console 中运行

```
monthly_counts <- bike |>
  dplyr::mutate(month = lubridate::floor_date(date, "month")) |>
  dplyr::count(neighborhood, month, name = "theft_count") |>
  dplyr::arrange(month, neighborhood)
```

逐行解释

`floor\_date(date, "month")` 把同月所有日期归到该月第一天，例如 2023-07-19 变为 2023-07-01。`count(neighborhood, month)` 等价于按社区和月份分组后统计行数；`name=` 给计数列命名。`arrange()` 只改变显示顺序，不改变数值。此处的计数就是响应变量，所以不能在 count 前删除事件。

运行后应该看到

`monthly\_counts` 每行对应一个确实发生过盗窃的社区月份；同一 neighborhood-month 组合只出现一次。

检查命令

```
monthly_counts |>
  dplyr::count(neighborhood, month) |>
  dplyr::filter(n != 1)
summary(monthly_counts$theft_count)
```

常见错误

若检查结果出现行，说明同一社区月份仍重复，通常是分组字段拼写不一致或没有真正执行 `count()`。不要把 `day\_of\_week` 一起加入分组，否则一行会变成“社区 × 月份 × 星期”，分析单位就变了。

评分连接

明确空间单位、时间单位和响应变量，可直接支持评分中的数据处理与研究问题一致性。

核心路径·照做与理解

第4步：补齐零事件月份并验证面板

这一步解决什么问题

加入没有记录但应当存在的社区月份，并把这些组合的盗窃数设为 0，形成规则的 140 × 120 面板。

为什么现在运行 没有事件的月份是 0，不是 NA。若忽略这些月份，平均值会偏高，趋势图断裂，模型也只学习“已经发生事件”的条件分布。

输入对象 `monthly\_counts` 与完整月份序列 `all\_months`。输出 `panel` 应有 16,800 行。

在 RStudio Console 中运行

```
all_months <- seq(
  as.Date("2014-01-01"), as.Date("2023-12-01"), by = "month"
)

panel <- monthly_counts |>
  tidyr::complete(
    neighborhood,
    month = all_months,
    fill = list(theft_count = 0L)
  ) |>
  dplyr::mutate(
    year = lubridate::year(month),
    month_of_year = lubridate::month(month),
    time_index = (year - 2014) * 12 + month_of_year
  )
```

逐行解释 `seq(..., by="month")` 生成连续 120 个月。`complete()` 展开所有社区和月份组合；`fill = list(theft\_count = 0L)` 只把新组合的计数补为整数 0。`year` 用于时间切分，`month\_of\_year` 用于季节特征，`time\_index` 把 2014-01 到 2023-12 编为 1 到 120，便于样条计算。

运行后应该看到 `panel` 有 16,800 行；每个社区恰好 120 行；`theft\_count` 没有 NA；`time\_index` 范围为 1-120。

检查命令

```
stopifnot(nrow(panel) == 140 * 120)
stopifnot(!anyNA(panel$theft_count))
stopifnot(all(table(panel$neighborhood) == 120))
range(panel$time_index)
```

常见错误 若行数不是 16,800，检查 `all\_months` 的起止日期、社区名称是否含前后空格，以及 `complete` 前是否仍有重复组合。若把 0 错补成 NA，后续 `mean()` 可能传播 NA 或通过 `na.rm=TRUE` 悄悄忽略月份。

评分连接 完整规则面板是时空可视化与基函数回归的共同数据基础，也是可复现性检查的重要证据。

核心路径·照做与理解

第5步：完成 Task 1 的四类时空图

这一步解决什么问题 分别回答长期变化、季节分布、空间热点和地点随时间变化四个问题，而不是用一张拥挤图承载所有信息。

为什么现在运行 可视化先提供结构证据：季节模式支持 Fourier，平滑长期变化支持 B-spline，持续热点支持空间基函数。

输入对象 `panel`，并从原始数据按社区计算中心经纬度后连接为 lon、lat。图形对象分别保存为 p_time、p_season、p_space、p_heat。

在 RStudio Console 中运行

```
monthly <- panel |>
  dplyr::group_by(month) |>
  dplyr::summarise(thefts = sum(theft_count), .groups = "drop")

p_time <- ggplot2::ggplot(
  monthly, ggplot2::aes(x = month, y = thefts)
) +
  ggplot2::geom_line(color = "#0B6E75") +
  ggplot2::labs(x = NULL, y = "Reported thefts")

ggplot2::ggsave(
  "output/figures/monthly_trend.png", p_time,
  width = 9, height = 5.4, dpi = 180
)
```

逐行解释

`group\_by(month)` 指定汇总单位，`summarise(sum(...))` 得到全市月总数。`ggplot(data, aes(...))` 建立数据与视觉属性映射：x 轴是月份，y 轴是数量；`geom\_line()` 决定用线连接；`labs()` 明确轴含义。`+` 是给图逐层添加组件，不是数值相加。`ggsave()` 保存指定图对象；width/height 是英寸，dpi 控制清晰度。季节箱线图、空间点图和时空热力图遵循同一语法层次。

运行后应该看到

输出四张图：总体月趋势、月份箱线图、社区热点图、前 25 高值社区热力图。每张图都应有标题、轴、单位和数据来源。

检查命令

```
print(p_time)
file.exists("output/figures/monthly_trend.png")
ggplot2::ggplot_build(p_time)
```

常见错误

出现 `object not found` 时，检查 monthly 是否先运行生成；出现空图时检查 aes 中的列名。空间图使用的是社区中心点，不要把它写成案件精确位置；比较多张地图时应固定颜色尺度。

评分连接

Task 1 占 30%。高分来自“每张图回答一个问题并推动模型设计”，而不是颜色多或图形复杂。

- 时间图：横轴时间、纵轴全市报告数；看趋势、突变和峰值。
- 季节图：每个箱表示同一月份跨十年的分布；看可重复季节性。
- 空间图：每个点是社区中心；颜色和大小表示长期月均数量。
- 热力图：一行一个社区、一列一个月；连续高色带表示持续热点。

其余三类 Task 1 图的核心图层

```
p_season <- ggplot2::ggplot(
  panel, ggplot2::aes(factor(month_of_year), theft_count)
) + ggplot2::geom_boxplot()

p_space <- ggplot2::ggplot(
  hotspots, ggplot2::aes(lon, lat,
    size = mean_monthly, color = mean_monthly)
) + ggplot2::geom_point() + ggplot2::coord_equal()

p_heat <- ggplot2::ggplot(
  heat, ggplot2::aes(month, neighborhood, fill = theft_count)
) + ggplot2::geom_tile()
```

其余三图为什么这样写

`factor(month\_of\_year)` 把 1-12 当作十二组而不是连续直线；`geom\_boxplot()` 比较每组跨年份分布。空间图把 lon/lat 映射到位置，把 mean_monthly 同时映射到大小和颜色，`coord\_equal()` 防止经纬方向比例失真。热力图用 `geom\_tile()` 让一个社区月份对应一个色块；正式图还要增加色觉友好色标、标题、图例和来源。

核心路径· 照做与理解

第6步：按时间划分训练、验证和测试集

这一步解决什么问题	模拟真实使用场景：用过去学习模型，用较新的年份选设置，最后用完全未见的 2023 年评价。
为什么现在运行	随机抽行会让同一社区的将来月份进入训练，造成时间泄漏。看似更高的分数不能代表未来预测能力。
输入对象	`panel`，其中 year 已从 month 可靠推导。输出 train、validation、test 三个互不重叠的数据框。

在 RStudio Console 中运行

```
train <- dplyr::filter(panel, year <= 2021)
validation <- dplyr::filter(panel, year == 2022)
test <- dplyr::filter(panel, year == 2023)
```

逐行解释	`filter()` 保留条件为 TRUE 的行；`<=` 表示小于等于，`==` 才是比较是否相等。2014–2021 用于估计结构，2022 只选择 λ ，2023 最后一次报告结果。不能写成 `year = 2023`，因为单个 `=` 在这里不是相等比较。
运行后应该看到	train 13,440 行，validation 1,680 行，test 1,680 行；年份分别为 2014–2021、2022、2023。

检查命令

```
stopifnot(max(train$year) < min(validation$year))
stopifnot(max(validation$year) < min(test$year))
c(train = nrow(train), validation = nrow(validation), test = nrow(test))
```

常见错误	若三个集合行数相加不等于 16,800，检查年份条件是否遗漏或重叠。不要用 `sample()` 随机拆分；也不要看到 2023 结果后不断修改模型再重测，否则 test 已经变成 validation。
评分连接	无泄漏验证、明确数据角色和诚实测试结果是 Task 4 方法可信度的核心加分点。

核心路径· 照做与理解

第7步：构造时空基函数与设计矩阵

这一步解决什么问题	把时间、月份、位置和社区名称转换为数值特征列，使线性模型能够表示非线性趋势、周期和空间差异。
为什么现在运行	回归只能读取数值矩阵 X。基函数负责把我们在 Task 1 看见的结构翻译成模型可以学习的列。
输入对象	`train` 以及只从训练期确定的 recipe：样条节点、空间中心、缩放尺度和社区水平。输出 x_train 数值矩阵。

在 RStudio Console 中运行

```
source("R/03_model.R")
splits <- split_panel(panel)
recipe <- make_basis_recipe(splits$train)

x_train <- make_design_matrix(splits$train, recipe)
x_validation <- make_design_matrix(splits$validation, recipe)
x_test <- make_design_matrix(splits$test, recipe)

# make_design_matrix() 内部的核心结构：
time_bs <- splines::bs(train$time_index, degree = 3,
  knots = recipe$time_knots,
  Boundary.knots = recipe$time_boundary)
season <- cbind(
  sin1 = sin(2*pi*train$month_of_year/12),
  cos1 = cos(2*pi*train$month_of_year/12)
)
area <- stats::model.matrix(~ neighborhood - 1, data = train)
```

逐行解释

`source()` 读取已经测试过的模型函数。`split\_panel()` 执行第 6 步的时间切分；`make\_basis\_recipe()` 只从训练集保存节点、空间缩放、16 个 RBF 中心和社区 levels。三次调用 `make\_design\_matrix()` 使用同一 recipe，保证列完全一致。函数内部：`bs()` 生成局部三次样条；`sin()`/`cos()` 生成年度周期；每个 RBF 计算 $\exp(-\text{distance}^2/(2*\sigma^2))$ ；`model.matrix(~ neighborhood - 1)` 为每个社区生成 0/1 列，`-1` 表示不自动加截距。

运行后应该看到

`x\_train` 的行数与 train 完全相同，列数远大于原始变量数；所有列都是数值。相同 recipe 可生成验证和测试矩阵。

检查命令

```
stopifnot(nrow(x_train) == nrow(train))
stopifnot(is.matrix(x_train))
dim(x_train)
x_train[1:3, 1:6]
```

常见错误

若验证或测试出现不同列数，通常是分别调用 `model.matrix()` 时因子水平不同。正确做法是保存训练期社区 levels，并把相同 levels 应用于未来数据。节点、中心和缩放参数不能从 test 的盗窃结果中选择。

评分连接

公式、设计矩阵与代码一一对应，能够证明你不是把模型当黑箱；这是方法解释深度的重要证据。

核心路径·照做与理解

第8步：建立两个简单预测基线

这一步解决什么问题

先回答“复杂模型是否比不建模更有价值”：比较全局历史均值和每个社区历史均值。

为什么现在运行

没有基线时， R^2 或误差数字缺乏参照。复杂模型即使看起来先进，也可能不如一个历史平均值。

输入对象

`train` 和 `test`。基线只能使用 train 计算均值，不能偷看 2022 或 2023。

在 RStudio Console 中运行

```
global_mean <- mean(train$theft_count)
pred_global <- rep(global_mean, nrow(test))

area_means <- train |>
  dplyr::group_by(neighborhood) |>
  dplyr::summarise(pred = mean(theft_count), .groups = "drop")
pred_area <- test |>
  dplyr::left_join(area_means, by = "neighborhood") |>
  dplyr::pull(pred)
```

逐行解释

`mean()` 得到训练期全局平均；`rep()` 为每个测试行重复同一预测。社区基线先 `group\_by()` 再求每个社区平均，`left\_join()` 按 neighborhood 把预测接回 test，`pull()` 抽出数值向量。连接前后必须保持测试行顺序。

运行后应该看到

得到长度均为 1,680 的 pred_global 和 pred_area；社区均值通常明显优于全局均值，说明稳定空间差异很重要。

检查命令

```
stopifnot(length(pred_global) == nrow(test))
stopifnot(length(pred_area) == nrow(test))
stopifnot(!anyNA(pred_area))
```

常见错误

若 pred_area 有 NA，说明测试集中出现训练未见社区，或社区名称不一致。不能用测试集自身均值填补；应说明外推限制，或退回全局均值/RBF-only 模型。

评分连接

与至少两个透明基线比较，可以直接支撑“复杂度带来额外价值”的论证。

核心路径· 照做与理解

第9步：拟合 OLS 基函数模型

这一步解决什么问题	用最小二乘估计设计矩阵每一列的权重，再把预测从 $\log(1+y)$ 尺度还原到盗窃数量。
为什么现在运行	OLS 是 Task 4 的直接实现，也是检验基函数本身是否有效的透明主模型。先理解它，再讨论正则化。
输入对象	`x_train_validation`、`y_train_validation` 和 `x_test`；最终拟合合并 2014–2022，预测 2023。

在 RStudio Console 中运行

```
train_validation <- dplyr::bind_rows(train, validation)
x_train_validation <- make_design_matrix(train_validation, recipe)
x_test <- make_design_matrix(test, recipe)
y_fit <- log1p(train_validation$theft_count)
ols <- stats::lm.fit(
  cbind(Intercept = 1, x_train_validation), y_fit
)
beta <- ols$coefficients
beta[is.na(beta)] <- 0
pred_ols_log <- cbind(Intercept = 1, x_test) %*% beta
pred_ols <- pmax(0, expm1(pred_ols_log))
```

逐行解释	`bind_rows()` 把训练集和验证集纵向合并； λ 已经确定后可利用 2014–2022 的全部历史信息完成最终拟合。`make_design_matrix()` 仍使用训练期 recipe。`log1p(y)` 精确计算 $\log(1+y)$ ，允许 $y=0$ 并缓解右偏。`lm.fit(X,y)` 直接对矩阵做最小二乘；`cbind(Intercept=1, ...)` 添加截距列。`%*%` 是高等代数中的矩阵乘法 XB 。`expm1()` 是 $\log 1p$ 的反变换，`pmax(0, ...)` 把不合理的负计数截为 0。共线列可能产生 NA 系数，本项目将其置零。
运行后应该看到	`pred_ols` 是长度 1,680 的非负预测；在真实 2023 测试上约得到 MAE 1.008、RMSE 2.048、 R^2 0.757。

检查命令

```
stopifnot(length(pred_ols) == nrow(test))
stopifnot(all(is.finite(pred_ols)))
stopifnot(all(pred_ols >= 0))
```

常见错误	若 `%*%` 报维度不匹配，检查训练和测试设计矩阵列数及列顺序。若忘记 `expm1()`，指标会把 $\log$ 尺度预测与原始计数比较，结果没有实际意义。OLS 系数很多时不应逐一作因果解释。
评分连接	这一步把理论公式 $y=XB+\epsilon$ 、实际 R 代码和测试集预测连成完整证据链。

核心路径· 照做与理解

第10步：拟合 Ridge 并选择 λ

这一步解决什么问题	给大系数增加平方惩罚，降低高度相关基函数造成的系数波动，并用 2022 验证集选择惩罚强度 λ 。
为什么现在运行	基函数和 140 个社区指示列可能相关。Ridge 是现代、稳健的扩展，但是否更好必须由验证结果决定。
输入对象	`x_train`、`y_train`、`x_validation` 和候选 <code>lambda_grid</code> 。2023 测试集完全不参与 λ 选择。

在 RStudio Console 中运行

```
lambda_grid <- exp(seq(log(100), log(1e-4), length.out = 100))
ridge_path <- glmnet::glmnet(
  x_train, log1p(train$theft_count),
  alpha = 0, lambda = lambda_grid, standardize = TRUE
)
val_log <- predict(ridge_path, newx = x_validation,
  s = lambda_grid)
val_rmse <- apply(val_log, 2, function(z) {
  pred <- pmax(0, expm1(z))
  sqrt(mean((validation$theft_count - pred)^2))
})
best_lambda <- lambda_grid[which.min(val_rmse)]
```

逐行解释

“glmnet()” 一次拟合多个 λ ；“alpha=0” 指 Ridge，惩罚为 $\lambda \sum \beta^2$ ；“standardize=TRUE” 先统一不同列的尺度。“predict(..., s=lambda_grid)” 给出每个候选 λ 的验证预测。“apply(..., 2,...)” 逐列计算 RMSE；“which.min()” 找最小误差位置。 λ 越大收缩越强，接近 0 时接近未惩罚回归。

运行后应该看到

本项目选择的 λ 约为 $1e-4$ ；Ridge 测试 RMSE 约 2.060，略差于 OLS 2.048。因此最终应诚实报告 OLS 略优。

检查命令

```
best_lambda
min(val_rmse)
stopifnot(best_lambda %in% lambda_grid)
```

常见错误

不要使用测试 RMSE 选择 λ ，也不要因为 Ridge 名字更新就宣布它获胜。若 λ 总落在网格边界，应扩大网格后只用验证集重选。对 glmnet 预测时 “newx” 必须是数值矩阵，不能直接传 tibble。

评分连接

展示超参数如何选择、并接受简单模型胜出，体现方法严谨性而不是模型宣传。

核心路径·照做与理解

第11步：计算 MAE、RMSE 和 R^2

这一步解决什么问题

用同一组 2023 实际值比较全部模型，并理解三个指标分别强调什么。

为什么现在运行

单一指标可能掩盖问题：MAE 表示典型绝对误差，RMSE 更惩罚大错， R^2 比较模型与测试集均值基准的平方误差。

输入对象

“actual = test\$theft_count” 与长度相同的预测向量 predicted。所有计算都在原始“每社区每月盗窃数”尺度。

在 RStudio Console 中运行

```
score <- function(actual, predicted) {
  tibble::tibble(
    MAE = mean(abs(actual - predicted)),
    RMSE = sqrt(mean((actual - predicted)^2)),
    R2 = 1 - sum((actual - predicted)^2) /
      sum((actual - mean(actual))^2)
  )
}
score(test$theft_count, pred_ols)
```

逐行解释

“actual-predicted” 是残差。“abs()” 消除正负抵消后取平均得到 MAE；平方后平均再开根号得到 RMSE，所以峰值错误权重更大。 $R^2=1-SSE/SST$ ；1 表示完全预测，0 约等于测试均值，负数表示比均值还差。 R^2 不是“预测正确百分比”。

运行后应该看到

全局均值：MAE 2.19、RMSE 4.17、 R^2 -0.007；社区均值：1.41、2.73、0.569；Basis OLS：1.01、2.05、0.757；Basis Ridge：1.01、2.06、0.754。

检查命令

```
stopifnot(length(test$theft_count) == length(pred_ols))
score(test$theft_count, pred_ols)
score(test$theft_count, pred_area)
```

常见错误

若出现极高误差，先确认预测是否已从 $\log$ 尺度反变换。若 R^2 为负不代表计算错误；它表示模型比测试均值基准更差。比较模型时必须使用完全相同的测试行，不能各自删除不同观测。

评分连接

指标表是 Task 4 结果页的核心证据；同时报告基线和多个指标比只给一个最好分数更可信。

核心路径·照做与理解

第12步：绘制预测与残差诊断

这一步解决什么问题

检查模型在哪些月份和社区系统性低估或高估，而不只盯住一个总体分数。

为什么现在运行

模型可能有不错 RMSE，却持续漏掉夏季峰值或中心区热点。残差结构说明哪些信息尚未被基函数解释。

输入对象

`test\_predictions`：包含 month、neighborhood、lon、lat、actual、predicted；新增 residual=actual-predicted。

在 RStudio Console 中运行

```
diagnostic <- test_predictions |>
  dplyr::mutate(residual = actual - predicted)

monthly_fit <- diagnostic |>
  dplyr::group_by(month) |>
  dplyr::summarise(
    actual = sum(actual), predicted = sum(predicted),
    .groups = "drop"
  )

residual_map <- diagnostic |>
  dplyr::group_by(neighborhood, lon, lat) |>
  dplyr::summarise(mean_residual = mean(residual), .groups = "drop")
```

逐行解释

`residual = actual-predicted`：正值表示模型低估，负值表示高估。月度汇总比较城市总量随时间变化；社区平均残差把 2023 的 12 个月压缩为每个地点一个诊断值。随后用 `ggplot()` 画实际/预测折线和以 0 为中心的发散色地图。

运行后应该看到

预测线应捕捉主要季节形状，但会低估部分夏季峰值；残差地图仍显示中心附近的正残差聚集。

检查命令

```
summary(diagnostic$residual)
mean(diagnostic$residual)
sum(diagnostic$residual > 0)
stopifnot(abs(diagnostic$residual - (diagnostic$actual - diagnostic$predicted)) < 1e-10)
```

常见错误

颜色方向很容易写反：必须在图注声明“正残差=实际高于预测”。地图残差不是犯罪原因；它只提示土地用途、交通、人流或暴露量等遗漏变量可能重要。

评分连接

诊断图把模型评价从“报分数”提升到“理解失败模式”，是高质量分析区别于机械建模的关键。

把诊断表变成预测图与残差地图

```
p_prediction <- ggplot2::ggplot(
  monthly_fit, ggplot2::aes(month)
) +
  ggplot2::geom_line(ggplot2::aes(y = actual,
    color = "Actual")) +
  ggplot2::geom_line(ggplot2::aes(y = predicted,
    color = "Predicted"))

p_residual <- ggplot2::ggplot(
  residual_map,
  ggplot2::aes(lon, lat, color = mean_residual)
) +
  ggplot2::geom_point() + ggplot2::coord_equal() +
  ggplot2::scale_color_gradient2(midpoint = 0)
```

为什么使用这些图层 两条 `geom\_line()` 共用 month 横轴，直接比较实际与预测的时间形状。`scale\_color\_gradient2(midpoint=0)` 强制残差色标以 0 为中点，使低估和高估使用相反方向颜色；正式图必须在图例中明确正负含义。

核心路径·照做与理解

第13步：把分析压缩为四页 PowerPoint

这一步解决什么问题 把完整分析转化为四页内可理解、可答辩的证据链，而不是把手册内容缩小后全部塞进去。

为什么现在运行 项目限制最多四页。必须选择最能回答研究问题的图和数字，省略不支持主线的中间输出。

输入对象 已验证的四张 Task 1 图、模型指标表、预测图、残差图及方法公式。

在 RStudio Console 中运行

```
slide_plan <- tibble::tribble(
  ~page, ~question, ~evidence,
  1, "研究什么？", "数据、研究问题、流程",
  2, "发现什么时空规律？", "趋势、季节、热点",
  3, "怎样建模并验证？", "基函数、时间切分、指标",
  4, "模型做到了什么？", "预测、残差、结论与局限"
)
slide_plan
```

逐行解释 `tribble()` 用逐行方式建立小表；`~page` 声明列名，后续每三项是一行。这里输入 R 代码不是为了自动生成 PPT，而是强迫自己检查每页只回答一个问题。页面 2 只保留最强的 Task 1 证据，页面 3 必须出现时间切分和基线。

运行后应该看到 得到 4×3 的展示计划；四页形成“问题→发现→方法→可信结论”的连续叙事。

检查命令

```
stopifnot(nrow(slide_plan) == 4)
stopifnot(length(unique(slide_plan$page)) == 4)
```

常见错误 不要复制完整代码到 PPT；代码属于复现材料，PPT 应展示方法结构和证据。字体不要小于 18pt，图标题直接写发现。不要把 0.757 写成“75.7% 盗窃被准确预测”，应写为测试 R^2 。

评分连接 表达与逻辑占 30%。四页各回答一个问题，比堆叠更多模型和图更容易拿到清晰度分数。

核心路径·照做与理解

第14步：从头复现并完成提交检查

这一步解决什么问题 验证一个空 R 会话能按照固定顺序重建审计表、面板、图形、模型指标和最终展示证据。

为什么现在运行 只在当前 Environment 中运行成功不等于可复现；对象可能来自之前某次试验。清空后重跑才能发现隐藏依赖和顺序错误。

输入对象

保存好的 `R/config.R`、`R/01\_prepare\_data.R`、`R/02\_visualize.R`、`R/03\_model.R` 和 `R/run\_all.R`，以及原始 CSV。

在 RStudio Console 中运行

```
rm(list = ls())
source("R/run_all.R")
list.files("output/analysis", full.names = TRUE)
list.files("output/figures", full.names = TRUE)
```

逐行解释

`ls()` 列出当前对象，`rm(list=...)` 清空 Environment，模拟第一次运行；这不会删除硬盘文件。`source()` 按顺序执行整个脚本。`list.files()` 检查输出是否真正写入磁盘。项目脚本应先创建目录、加载包，再读取数据、作图和建模。

运行后应该看到

自动生成 data_audit.csv、bicycle_monthly_panel.csv、model_metrics.csv、test_predictions.csv，以及 8 张图；控制台无错误。

检查命令

```
stopifnot(file.exists("output/analysis/model_metrics.csv"))
stopifnot(length(list.files("output/figures", pattern = "\\png$")) >= 8)
readr::read_csv("output/analysis/model_metrics.csv", show_col_types = FALSE)
```

常见错误

若清空后报 `object not found`，说明脚本依赖了未在文件中创建的对象；把生成该对象的代码放到正确的前序脚本。不要在脚本中写 `setwd()` 指向个人绝对路径，也不要把测试集结果用于重新选择模型。

评分连接

最终提交前的可复现运行同时保护数据处理、方法、结果和表达四条证据链。确认 PPT 不超过四页，并保留脚本和输出作为答辩依据。

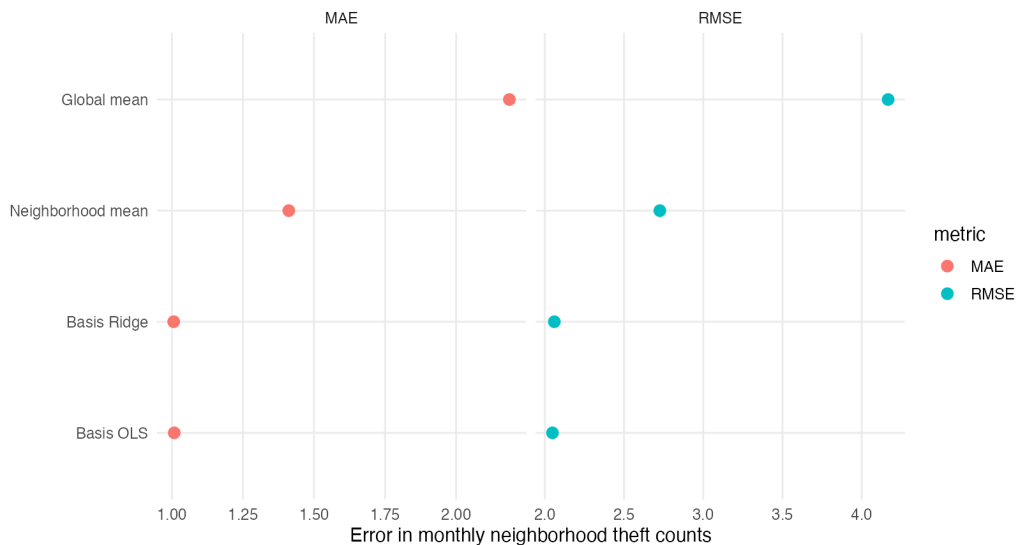
核心路线

真实模型结果与解释

全局均值	2.19	4.17	-0.007
社区历史均值	1.41	2.73	0.569
基函数OLS	1.01	2.05	0.757
基函数Ridge	1.01	2.06	0.754

Out-of-time test performance

All scores use 2023 only; lower MAE and RMSE are better



2023时间外测试：基函数模型明显优于两个均值基线。

基函数OLS相对社区均值基线把RMSE降低约25%，说明季节、长期趋势和空间/社区结构提供了历史均值之外的信息。Ridge没有进一步提升，说明当前设计矩阵在本任务上已较稳定。

怎样写结论

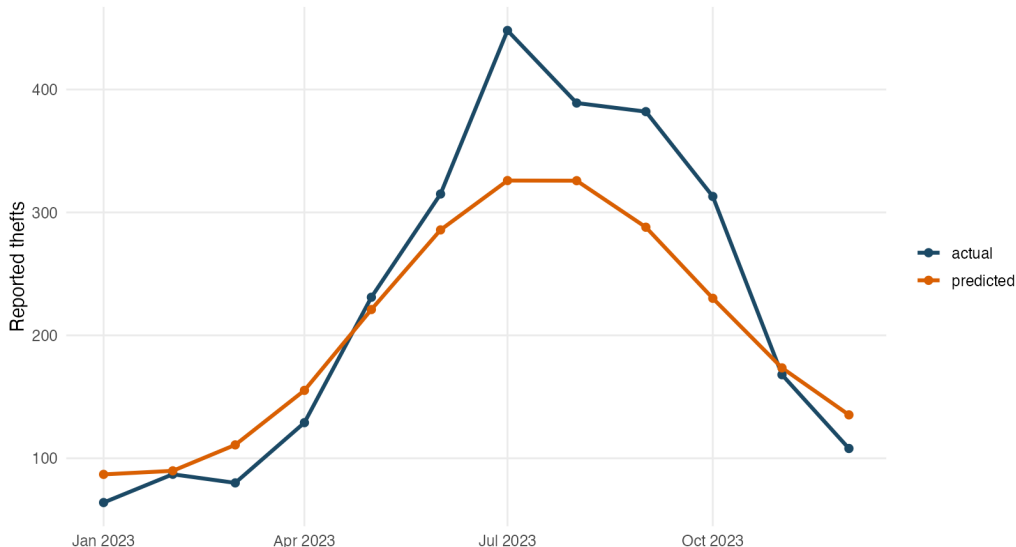
可以说“模型在2023测试集上具有更低预测误差”，不能说“这些变量导致盗窃”，也不能把测试 R^2 解释为对所有城市和未来年份都同样有效。

进阶加分

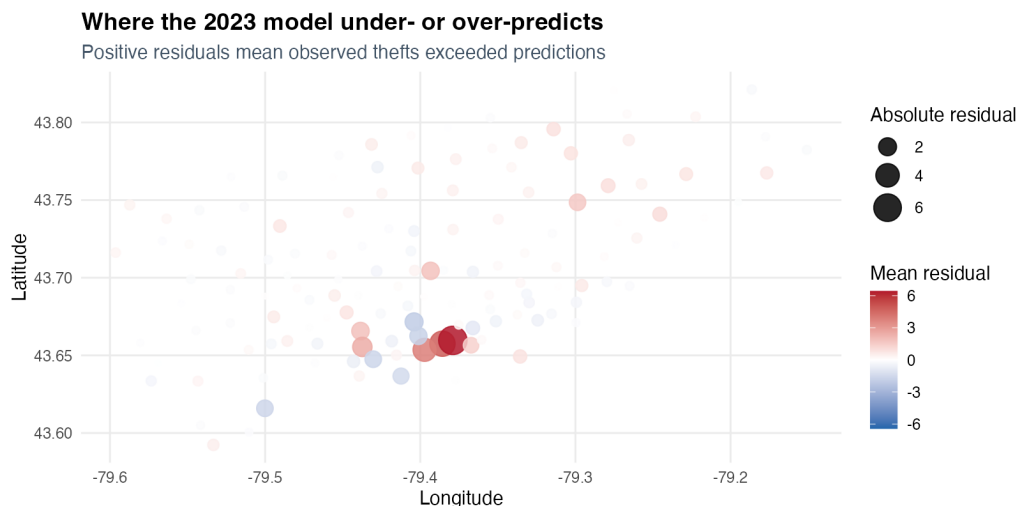
预测图、残差图与模型诊断

Observed and predicted citywide totals in 2023

Predictions are summed from neighborhood-level Ridge estimates



2023总体预测捕捉了季节形状，但低估了夏季峰值。



正残差表示实际盗窃高于预测，揭示模型仍未解释的局部热点。

- 峰值低估是平方误差模型和平滑基函数常见现象。
- 残差仍在中心区域聚集，说明土地用途、交通、人流等遗漏变量可能重要。
- 残差图不只是装饰；它检查模型是否系统性遗漏空间结构。
- 进一步可检查残差随时间、拟合值和社区的分布，但四页PPT只保留最关键证据。

核心路线

四页PPT逐页设计

1	问题与数据	研究问题、31,833事件、140社区、120个月、工作流
2	Task 1发现	趋势图、热点图、季节性与描述性发现
3	Task 4方法	基函数公式、训练/验证/测试、模型比较
4	预测与结论	预测/残差图、三条结论、局限和下一步

- 每页只回答一个问题，不把所有诊断都塞进展示。
- 字体至少18pt；图标题直接写发现，不只写变量名。
- 模型页必须出现简单基线和无泄漏时间切分。
- 结论页必须同时说模型做到了什么、没做到什么。

推荐叙事

看见明显季节与热点 → 用对应基函数编码结构 → 用过去预测2023 → 比基线更准但低估夏季峰值 → 得出有限、可信的结论。

进阶加分

答辯问题与推荐回答

为什么不随机拆分？	会把未来或相邻月份泄漏进训练，误差过于乐观。
为什么 $\log(1+y)$ ？	缓解右偏和极端峰值；预测后还原到计数尺度评价。
为什么既有RBF又有社区指标？	RBF学连续空间，社区指标学稳定但非平滑的地点差异。
Ridge为什么没赢？	验证选择接近零惩罚，说明当前任务无需强收缩；如实报告。
能否预测新社区？	包含社区指标的主模型不能直接外推；需用RBF-only空间模型。
这是因果模型吗？	不是；它描述和预测报告数量，未控制暴露量和混杂因素。

核心路线

局限性、伦理与不可推广范围

- 事件无唯一ID：1,475条完全相同行可能是真实多起事件，也可能含重复记录。
- 坐标是社区中心点：不能解释为案件发生的精确位置。
- 缺少暴露量：高数量可能部分来自人口、自行车拥有量或活动量更高。
- 报案偏差：数据描述被报告事件，不一定覆盖所有真实盗窃。
- 模型只测试2023：不能保证制度、疫情或城市结构变化后的稳定性。
- 热点展示可能造成社区污名化：使用中性语言，不把地点居民描述为原因。

正确措辞

写“报告的自行车盗窃数量在这些社区较高”，不要写“这些社区更危险”或“社区导致盗窃”。

未来连接

从本项目走向未来数据科学

基函数与线性模型	GAM、P-spline、趋势面回归	特征含义与平滑度验证
Ridge与验证	Lasso、Elastic Net、Boosting	超参数只用验证集
社区月份面板	随机森林、XGBoost	时空拆分和泄漏检查
空间RBF	Gaussian Process、Kriging	距离、尺度和不确定性
时间基函数	状态空间、LSTM	过去预测未来
残差地图	空间误差模型、GNN	检查遗漏的结构

真正可迁移的不是某个包名，而是：定义粒度、构造特征、建立基线、隔离测试集、选择指标、诊断残差、限制结论。这些原则在传统统计、机器学习和深度学习中都不会过时。

核心路线

从开始到提交的最终检查清单

- 研究问题能否用一句话说清？
- 一行数据代表什么？时间和空间单位是否明确？

- quarter是否从date重新计算？重复行是否说明而非武断删除？
- 四张Task 1图是否分别回答时间、季节、空间和时空问题？
- 每类基函数是否有明确原因？
- 训练、验证、测试是否严格按时间分开？
- 是否与至少两个简单基线比较？
- MAE、RMSE和 R^2 是否来自2023测试集？
- 是否展示预测和残差，而不只报最好分数？
- 结论是否避免因果化、风险化和过度推广？
- 四页PPT是否每页只有一个中心问题？
- R脚本能否从头运行并重新生成所有图表？

未来连接

资料与继续学习入口

- 原始任务：project.pdf。
- 原始数据：four_dataset/bicycle.csv。
- 本地知识库：矩阵与线性变换、特征值与特征向量、协方差与相关系数、PCA与SVD、R与时空数据入门、ggplot 2与空间地图。
- R官方：splines::bs与predict.bs文档。
- R官方：mgcv smooth terms文档。
- tidymodels：sliding_window、sliding_index和sliding_period时间重采样。
- spatialsample：空间分块与leave-location-out验证。
- StatQuest：PCA、回归、正则化和模型评价视频。

学习顺序

先能解释设计矩阵和验证，再深入样条推导；先能复现实验和诊断，再追求更复杂模型。